

Lesson 9: Working with JSON Data

Where app data comes from

Most real apps show data that comes from somewhere else: a server, a file, or an API. That data almost always arrives as JSON -- a simple text format for structured information.

JSON looks like this:

```
{
  "name": "Aarav Sharma",
  "age": 21,
  "isStudent": true
}
```

It is just keys and values inside curly braces: strings in quotes, numbers plain, true/false for yes/no. A list of users would be an array of such objects:

```
[
  { "name": "Aarav Sharma", "age": 21 },
  { "name": "Diya Patel", "age": 22 }
]
```

If you can read this, you can read JSON. There is nothing more to it.

From JSON to Dart: model classes

In Dart, you do not want to pass raw JSON maps around your app -- it is easy to mistype a key and get a crash. Instead, you convert each JSON object into a model class: a simple class that holds the data with named fields.

```
class User {
  final String name;
  final int age;

  User({required this.name, required this.age});
}
```

Then you add a factory constructor called fromJson that builds a User from a JSON map:

```
class User {
  final String name;
  final int age;

  User({required this.name, required this.age});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(name: json['name'], age: json['age']);
  }
}
```

```
factory User.fromJson(Map<String, dynamic> json) {
  return User(
    name: json['name'],
    age: json['age'],
  );
}
```

factory just means "a constructor that does some work before returning the object." Now `User.fromJson({'name': 'Aarav', 'age': 21})` gives you a proper `User` object, and the rest of your app never touches raw JSON.

Showing a list of models

Once you have a list of `User` objects, displaying them is just Lesson 7 again:

```
final users = [
  User(name: 'Aarav Sharma', age: 21),
  User(name: 'Diya Patel', age: 22),
];

ListView.builder(
  itemCount: users.length,
  itemBuilder: (context, index) {
    final user = users[index];
    return ListTile(
      title: Text(user.name),
      subtitle: Text('Age ${user.age}'),
    );
  },
)
```

Models and lists fit together naturally: the model holds the data cleanly, the list shows it.

Loading data with FutureBuilder

Real data takes time to arrive -- it comes from the network or a file. While waiting, you show a loading spinner; when it arrives, you show the list. `FutureBuilder` handles exactly this pattern.

You give it a future (the slow work) and a builder (what to show for each stage):

```
FutureBuilder<List<User>>(
  future: fetchUsers(), // returns Future<List<User>>
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(child: CircularProgressIndicator());
    }
    if (snapshot.hasError) {
```

```
        return const Center(child: Text('Something went wrong'));
    }
    final users = snapshot.data!;
    return ListView.builder(
      itemCount: users.length,
      itemBuilder: (context, index) {
        return ListTile(title: Text(users[index].name));
      },
    );
  },
)
```

Three stages, three branches:

- Waiting -- the data has not arrived yet: show a spinner.
- Error -- something failed: show a friendly message.
- Done -- snapshot.data holds your list: show it.

You will learn how `fetchUsers()` itself works (async, HTTP) in Lessons 10 and 11. For now, remember the shape: spinner, error, data.

Key terms

- JSON -- a simple text format for structured data: keys, values, and arrays.
- Model class -- a Dart class that holds one item of data with named fields.
- `fromJson` -- a factory constructor that builds a model object from a JSON map.
- `FutureBuilder` -- a widget that shows different UI while data loads, fails, or arrives.
- `snapshot` -- what `FutureBuilder` gives your builder: the current state of the loading work.